

Introduction to Computer Science

Fall 2026

Homework 1

Due: September 27, 2026 at 23:58:59
No Copy-and-Paste. No Late Submission.

Instructions. Please write your answers to each problem using L^AT_EX and submit the compiled PDF file. If you are unfamiliar with L^AT_EX, you may refer to <https://latex-tutorial.com/> or other tutorials for help.

If you find a problem difficult to understand or solve, you may use external resources, including search engines (such as Google), online references (such as Wikipedia), and AI tools (such as DeepSeek), to study relevant concepts and assist your learning. However, you are responsible for understanding your submitted solution, and **please make sure every word in your submission is written by yourself to avoid plagiarism.**

If you use any AI tool while working on the assignment, please include a short disclosure at the end of your submission stating which tool you used and briefly describing how you used it. Undisclosed use of AI tools may be subject to point deductions.

1 Is This Shuffle Random? (3 points)

Random shuffling appears in many places in computer science and everyday applications. For example, we may want to shuffle a deck of cards in a game, randomize the order of students in an experiment, or randomly permute data before training a machine learning model. In all these cases, we would usually like every possible ordering to be equally likely.

Consider the following algorithm for shuffling n elements, where $n \geq 3$.

Algorithm 1 Shuffling n elements

```
1: initially  $\{a_k\} = \{k\}$ , where  $k \in \{1, 2, \dots, n\}$ 
2: for  $i = 1$  to  $n$  do
3:   choose  $j$  uniformly from  $\{1, 2, \dots, n\}$ 
4:   swap  $a_i$  and  $a_j$ 
```

Does this algorithm generate all $n!$ permutations with equal probability? Justify your answer.

2 Transformer (3 points)

In this week's lecture, we discussed several representative achievements made possible by large language models (LLMs). Behind many of these systems is a fundamental architecture called the **Transformer**.

Transformer [1] is such an important natural language processing model that even today's large models are still based on the Transformer architecture. The Transformer does not process raw text; it processes a sequence of *tokens*, where a token is just a small piece of text, such as a word or a punctuation mark.

- (a) A core idea in Transformers is embedding: mapping discrete tokens to continuous vectors. Let $V = \{v_1, \dots, v_n\}$ be a vocabulary of $n = 50000$ distinct tokens, and let each token v_i is a d -dimensional

vector. With $d = 768$, assume each token embedding is drawn uniformly from $[-1, 1]$ for simplicity. For any two distinct token embeddings v_i and v_j , what are the expectation and variance of

$$\cos(v_i, v_j) = \frac{v_i^\top v_j}{\|v_i\| \|v_j\|}?$$

- (b) Self-attention is a core component of the Transformer. For simplicity, we use a simplified version of it here.

When a passage of n tokens is fed into a Transformer, let its i -th token be represented by an embedding $x_i \in \mathbb{R}^d$. The layer has three learned matrices $Q, K, V \in \mathbb{R}^{d \times d}$, and

$$q_i = Qx_i, \quad k_i = Kx_i, \quad v_i = Vx_i.$$

For every pair i, j , define

$$c_{ij} = q_i \cdot k_j = \sum_{r=1}^d q_{i,r} k_{j,r}.$$

Then define

$$y_i = \sum_{j=1}^n c_{ij} v_j.$$

Define one operation as one multiplication of two real numbers or one addition of two real numbers. Compute how many operations are needed in total to obtain $y_1, \dots, y_n$ from $x_1, \dots, x_n$, and express the answer as a function of n and d .

- (c) Suppose a Transformer has $L = 100$ layers, and each layer performs the computation described in part (b). On average, each query contains $n = 10^5$ tokens, with embedding dimension $d = 10^4$. Assume that each addition or multiplication consumes 2×10^{-10} J of energy. If 7×10^9 people each submit one such query per day, estimate the total energy consumed by the Transformer per day.

3 Quantum Key Distribution (4 points)

In this week's lecture, we discussed **quantum key distribution** as a form of provable security based on the laws of quantum mechanics. In this problem, we will explore why eavesdropping can be detected.

Alice and Bob would like to establish a shared secret key using the **BB84 quantum key distribution protocol**. Unlike a classical bit, measuring a qubit may change its state.

A **qubit** can be represented by a normalized vector

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle, \quad |\alpha|^2 + |\beta|^2 = 1,$$

where

$$|0\rangle = \begin{bmatrix} 1 \\ 0 \end{bmatrix}, \quad |1\rangle = \begin{bmatrix} 0 \\ 1 \end{bmatrix}.$$

In this problem we use four states and two measurement bases:

$$|0\rangle, \quad |1\rangle, \quad |+\rangle = \frac{|0\rangle + |1\rangle}{\sqrt{2}}, \quad |-\rangle = \frac{|0\rangle - |1\rangle}{\sqrt{2}},$$

$$Z = \{|0\rangle, |1\rangle\}, \quad X = \{|+\rangle, |-\rangle\}.$$

To measure a qubit in a basis, first express the state as a linear combination of the two basis states. If

$$|\psi\rangle = \alpha|b_0\rangle + \beta|b_1\rangle,$$

then measuring in the basis $\{|b_0\rangle, |b_1\rangle\}$ gives outcome $|b_0\rangle$ with probability $|\alpha|^2$ and $|b_1\rangle$ with probability $|\beta|^2$. After the measurement, the qubit is left in the state corresponding to the observed outcome.

3.1 Establishing a shared key

Alice chooses a random bit and independently chooses either the Z or X basis. She encodes the bit as

Classical bit	Z basis	X basis
0	$ 0\rangle$	$ +\rangle$
1	$ 1\rangle$	$ -\rangle$

Bob independently chooses either Z or X and measures the received qubit. Alice and Bob then publicly announce their bases and keep only the positions where their bases agree. This is called **basis sifting**.

- (a) Suppose Alice and Bob choose between the X and Z bases independently and uniformly at random.
- 1) Show that, in the absence of noise and eavesdropping, Bob's classical bit is identical to Alice's bit at every retained position.
 - 2) If Alice initially transmits N qubits, approximately how many shared random bits do Alice and Bob obtain?

The remaining shared bit string is called the **sifted key**.

3.2 What happens when Eve listens?

Suppose Eve intercepts every qubit. For each qubit, she independently chooses the X or Z basis uniformly at random, measures it, and sends the state she observed to Bob. This is called an **intercept-resend attack**.

For the following questions, consider only positions that Alice and Bob keep after basis sifting.

- (b) Compute the expected error rate among the bits in Alice and Bob's sifted key.
- (c) To detect eavesdropping, Alice and Bob randomly select m bits from their sifted key and publicly compare them. Assume that the communication channel is otherwise noiseless and that errors at different positions are independent. Using your result from part (b), derive the probability that Eve is detected. What happens to this probability as m increases?

Remark. In this problem, we have analyzed a particular intercept-resend attack. This is not a complete security proof of BB84; proving security against arbitrary quantum attacks requires additional ideas beyond the scope of this assignment.

Declaration of AI Use

Please disclose any use of generative AI tools in completing this homework. For each tool used, briefly state what you used it for. You remain responsible for the correctness and originality of your submission. If you did not use any generative AI tools, remove this section.

References

- [1] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.